

Candidate changes: behavior, safety evidence and limits

This is an experiment/design document, not approval to deploy a product change. The final report distinguishes measured kernel endpoint lifecycles from the Nullnet control-plane lifecycle. A throughput number is not evidence of isolation.

1. Reuse fully reset dedicated bridges from a bounded pool

Difference. Keep the existing one-bridge-per-endpoint forwarding topology. A clean anonymous bridge is leased and renamed to the endpoint's normal bridge name. Teardown deletes its transport and endpoint links, removes crypto, puts the bridge DOWN, removes all IPv4/IPv6 addresses and neighbors, and returns it under an anonymous pool name. The pool has a fixed capacity; an active lease is never allocated twice. Cold bridge creation and eventual destruction of the whole pool are separate measured costs.

Why it preserves the tested packet behavior. There is no shared L2 forwarding domain, VLAN filtering, tag rewriting or bridge-netfilter sysctl change. The gateway address stays on an actual bridge with the same active interface name. Current and candidate both pass exact delivery of eight Ethernet formats, including nested tags and priority bits; gateway reachability; unchanged FORWARD drop enforcement; and source-policy routing through two stages of SNAT using per-gateway interface rules. Independent MACsec/IPsec keys remain per edge.

Reset evidence. Idle slots must be DOWN, anonymous and free of addresses, neighbors, dynamic FDB entries, multicast memberships and endpoint ports. The lifecycle trials assert these inventories. Packet reuse tests seed a permanent neighbor and multicast membership before retiring the edge, then recreate it with fresh keys. Concurrent churn, partial-crypto failure and container restart exercise reuse and survivor isolation. The bridge MAC is explicitly pinned and independently generated across hosts: the selected configuration uses locally administered addresses that remain stable during each pool lifetime.

Ownership requirements. Production must carry a lease generation through setup, rollback and teardown, use the existing lifecycle serialization, and return a slot only after all cleanup ACKs. On restart, reconcile owned active leases before adoption; never adopt a Docker-owned bridge. Keep the current forwarding/multicast settings. Bridge kernel counters survive reuse; any future per-lease counter export must subtract a lease baseline. Current Nullnet client code does not export bridge rx/tx counters.

Cost and limitation. This removes bridge destruction from endpoint retirement; it does not make kernel bridge destruction fast. Removing the entire 1,000-slot pool still takes about 19–20 seconds. The report includes that cost explicitly. A workload that requires returning to zero bridge objects after every burst does not meet the requested teardown rate. An endpoint retirement does leave no endpoint identity, ports, addresses, neighbor state or keys behind; only an anonymous reusable bridge remains.

2. Share the VXLAN UDP destination port while keeping one VXLAN interface and VNI per edge

Difference. VXLAN devices share a compatible kernel UDP socket. Dedicated devices/VNIs remain. The selected prototype uses reserved UDP port 4790. Port reservation must be explicit; applying its fail-closed guard to Docker Swarm port 4789 would interfere with unrelated traffic.

Why it matters. The earlier bare trace proved that each last-user socket release incurs a normal `synchronize_rcu()` wait. One socket per edge produced roughly 20 seconds per 1,000 VXLAN deletions; one shared socket reduced the bare device-only deletion to roughly 0.2 seconds. Full endpoint teardown also contains the separate bridge cost above.

Safety condition. The current XFRM policies distinguish edges using destination ports. A shared-port change MUST be coupled to an encryption scheme that still selects and validates the correct edge. Simply changing the port is invalid.

3. Marked per-edge IPsec with a shared VXLAN port

Difference. Set a trusted per-edge mark before VXLAN encapsulation; choose outbound XFRM policy/state by that mark. Restore the edge mark from the authenticated inbound SA, and validate it against the receiving VXLAN device. Keep independent directional keys/SAs and a fail-closed rule when XFRM state/policy is absent.

Why it preserves the tested encryption behavior. IPsec still protects the VXLAN header, including VNI. Authenticated SA identity must match the receiving VNI; a successful ping alone does not prove this binding. An ordinary XFRM selector has no VNI field. Marks must survive the exact namespace and encapsulation path, and must be assigned/validated by trusted host code.

Observed evidence. The final native prototype passed 32 Docker-backed edges in both directions on the real hosts, wrong-key isolation, out-of-window ESP replay rejection, three fresh-key recreation cycles, and total-crypto-loss fail-closed capture. Deliberately sending VNI 1 through edge 0's valid SA was rejected by the receiving ingress filter; edge 0 survived and restoring the correct mark recovered edge 1. The selected after timings include every qdisc, filter, state and policy operation. The final report gives repeated matched current-versus-candidate TCP measurements and small-packet UDP loss. That tradeoff and 1,000-active-edge dataplane scaling require further integrated validation. Shared BPF-map filtering is not implemented or claimed as an additional gain.

4. Replace MACsec subprocesses with native route + generic Netlink

Difference. Send MACsec device creation through `NETLINK_ROUTE` and TX SA/RX channel/RX SA updates through the macsec generic-Netlink family. Retain the same order and check every ACK. This replaces four `ip` launches, or one `ip -batch` launch, per endpoint.

Why it can be equivalent. The experiment follows the actual Linux UAPI and `iproute2` source, including 64-bit cipher ID, 16-byte key ID, 32-byte AES key, association number, PN and SCI. In particular `IFLA_MACSEC_PORT` is encoded in network byte order: the existing `rtnetlink .port(1)` endianness bug must NOT be reintroduced. A normal Netlink ACK does not contain a newly allocated interface index.

Validation. Compare normalized kernel configuration and actual encrypted delivery against `iproute2`, not just successful ACKs. Batching/native Netlink is not atomic: partial configuration still requires cleanup.

Observed evidence. The native path carries actual encrypted traffic in the production-style same-host /29 layout. On both hosts it passes bidirectional traffic, MTU, TX-SA disable/restore, five fresh-key reuse cycles, unrelated-edge survival and benchmark-container restart recovery. The selected after includes all three generic-Netlink operations and device creation. Replay is explicitly enabled at window 128, whereas current setup defaults differ; this is a security strengthening with a finite reordering window, not byte-for-byte configuration equivalence.

5. Replace namespace `nsenter ip` subprocesses with namespace-bound Netlink sockets

Difference. A dedicated OS thread enters the selected network namespace long enough to open a Netlink socket, then restores its original namespace. The socket remains bound to the target namespace and configures the endpoint address/MTU/UP directly.

Why it can be equivalent. It sends the same route-Netlink operations in the same order to the same namespace. No async-runtime thread may call `setns`. Preserve standalone default routes and Docker-owned routes. Namespace/socket FDs must be closed after ownership ends.

Container identity condition. Removing per-endpoint Docker inspection requires an authoritative discovery entry, container ID/start generation and a pinned namespace FD. A forever PID cache is unsafe under container restart, name reuse, PID reuse and watcher gaps. Refresh/invalidate on events, reconcile watcher reconnect, and validate generation before publishing a setup. The refreshed same-host proof restarts a benchmark container, rejects the old pinned pidfd generation before creating a veth, refreshes namespace/pid handles and restores connectivity. This is evidence for the identity mechanism, not an implementation of production Docker event/watch reconnect handling. The namespace-bound sockets are opened on ordinary dedicated worker threads, not async runtime threads.

6. Set forwarding policy once, with reconciliation

Difference. Remove repeated `sysctl ip_forward=1` and `iptables -P FORWARD ACCEPT` from each endpoint setup after startup establishes them.

Why it can preserve behavior. These are host/namespace-global settings, not endpoint state. The previous benchmark proved repeated `nftables` transactions dominated setup. But existing startup code, Docker/firewall reloads and failure diagnostics must be reconciled; "once" cannot mean silently assuming the setting never changes. No broader firewall policy is proposed.

7. Change deletion batch size / native worker count only from measured evidence

Difference. Amortize Netlink dumps, regrouping and generic device-unregistration costs while bounding the time deletion holds RTNL. Preserve per-edge setup/teardown serialization and acknowledged release of IDs/bridge leases.

Safety condition. Only retiring owned devices enter the delete group. Explicitly delete owned links and await ACKs before namespace unmount or ID reuse; the harness reproduced EEXIST when it relied on asynchronous namespace destruction alone. Keep other edges, Docker interfaces and shared infrastructure out of it. Test cancellation, partial deletion, concurrent creation, reuse and final cleanup. Report setup tail latency during deletion as well as aggregate deletion throughput. A huge delete batch can improve an isolated benchmark while starving new setups.

Observed evidence. The selected dedicated-bridge candidate tests batch 256 on both hosts and both encryption modes, maintaining 512 live endpoint halves and swapping 256 per wave for six waves, with clean bridge return and survivor checks. The report distinguishes active-operation p99 from burst completion including admission, and records deletion ACK duration. The selected idle burst uses 256, retaining the previously chosen 32-worker/eight-command limits. This does not authorize replacing the product's active-setup batch policy without an integrated latency budget. No claim is made that individual RTNL hold time equals the whole deletion batch wall time.

Primary source anchors

- Linux bridge multicast destructor: https://github.com/gregkh/linux/blob/v6.12.95/net/bridge/br_multicast.c
- VXLAN socket sharing/release: https://github.com/gregkh/linux/blob/v6.12.95/drivers/net/vxlan/vxlan_core.c
- MACsec UAPI/implementation: https://github.com/gregkh/linux/blob/v6.12.95/include/uapi/linux/if_macsec.h and <https://github.com/gregkh/linux/blob/v6.12.95/drivers/net/macsec.c>
- iproute2 MACsec configuration: <https://github.com/iproute2/iproute2/blob/v6.12.0/ip/ipmacsec.c>
- XFRM selector/mark API: <https://github.com/gregkh/linux/blob/v6.12.95/include/uapi/linux/xfrm.h>

8. Native XFRM and TC configuration for the marked-IPsec candidate

Difference. NETLINK_XFRM creates/deletes two directional states and one marked outbound policy per edge directly. The selected variant initializes one shared inbound policy per peer/port and removes it only after its final owner is gone. NETLINK_ROUTE creates the VXLAN's clsact qdisc, outbound trusted-mark action, inbound matching-mark accept filter and default drop filter. These are the same kernel APIs exercised by the working `ip xfrm / tc` prototype, without a subprocess per operation.

Why it can preserve behavior. The candidate still uses per-edge AES-256-GCM transport-mode ESP and keeps the encrypted VXLAN header. State and policy updates use explicit addresses, SPIs and marks; teardown removes only that edge's objects. Inbound SA output marks are local trusted metadata, not marks accepted from the remote VXLAN header. The receiving interface must match the authenticated mark. The fault test deliberately encrypts VNI 1 under edge 0's valid SA; edge 1 must reject it, edge 0 must continue, and restoring the correct outbound mark must recover edge 1.

Publication order. Create the VXLAN down and unattached. Install trusted egress marking, inbound accept/drop filters and both directional states/policies before exposing it to its endpoint bridge. The fail-closed port guard must already exist. On teardown, remove exact owned crypto and links, await ACKs, then release bridge-lease/mark/VNI ownership. A failed partial setup must never be published as ready.

Additional safeguards. Direction-specific derived keys avoid sharing key material between directional SAs. The final XFRM candidate explicitly uses a 4,096-packet replay window; the finite window accommodates reordering while rejecting old captured ciphertext. See change 12. A host/namespace-level rule rejects unprotected UDP on the selected VXLAN port even if XFRM policies disappear. This rule must be installed before exposing endpoints, ordered before any rule or offload path that could bypass it, and survive firewall reconciliation. Appending it blindly to an existing production ruleset is not sufficient; the isolated test namespace had no competing ACCEPT rules. Missing state with a required policy must also fail closed.

ABI caveat. The Python experiment encodes the verified Linux IPv4 UAPI layouts and checks message ACKs. This is evidence for the design, not production-ready Rust serialization. Production code must use checked layout/byte-order handling and pass configuration-equivalence and packet tests. This configuration is not atomic; partial failures need per-generation cleanup before an endpoint is published.

Applying these proposals

Each numbered section is also exported as an individual Markdown document under changes/. These documents explain conditions under which a change can be safe; none claims an unimplemented Nullnet change has passed its release gates. Production integration must preserve TLS control, firewall bindings, routing/proxy behavior, graph state, client restart recovery, lifecycle locks and diagnostics. The selected after is a tested Linux endpoint design, with limits stated in analysis.md.

9. Native creation and removal of standalone network namespaces

Difference. A dedicated OS thread saves its network namespace FD, creates a new network namespace with `unshare(CLONE_NEWNET)`, bind-mounts `/proc/thread-self/ns/net` at the owned named path, and restores the original namespace in a finally block. Removal uses `umount2(MNT_DETACH)` and `unlink` after endpoint links have been deleted. This replaces one `ip netns` process per creation/removal and preserves a fresh namespace for each incarnation.

Why it can preserve behavior. These are the same core operations used by `iproute2`. The `/run/netns` mount directory must first be initialized and made shared exactly once under the startup lock; the experiment's outer `ip netns` fixture provides that initialization. Use the calling thread's namespace path, never `/proc/self/ns/net` on a multithreaded client. Treat a namespace-restore failure as fatal to that worker. Do not recycle dirty namespaces or move arbitrary `async-runtime` threads.

Observed evidence and limit. Both hosts passed three 1,000-endpoint same-host and cross-host trials, native default-route installation, endpoint-to-gateway ping, exact owned-object cleanup and absence of the namespace mount paths afterward. This reduces process overhead, especially removal, but standalone full-sequence throughput remains below the 600+/s goal. This is a measured extra candidate, not grounds to claim the Docker rate for standalone endpoints.

[Actual iproute2 namespace implementation.](#)

10. Share only the inbound IPsec policy while keeping independent SAs

Difference. One inbound policy per host pair and reserved UDP port requires transport-mode ESP without pinning a single SPI. Each edge still has independent directional SAs and keys, an outbound marked policy and a receiving VXLAN filter that accepts only its authenticated SA output mark.

Why it can preserve isolation. Encryption authentication is provided by the per-edge SA. The shared inbound policy requires authenticated ESP from the expected host pair; the VXLAN ingress check subsequently binds that authenticated SA identity to the receiving edge. Sharing a selector policy does not share key material. The ingress identity check and fail-closed port guard become essential invariants and must precede publication. Exact state cleanup must not remove the shared policy while another edge still depends on it.

Observed status. The final combined configuration is checked at 1,000 live cross-host edges with 2,000 directional pings and first/last-edge TCP samples. The combined 32-edge proof passed wrong-key, wrong-SA/VNI, old-frame replay, missing-all-crypto, VLAN tag isolation and fresh-key recreation tests. Its lifecycle and 1,000-edge traffic measurements are reported separately.

Why not share outbound too. Linux `xfrm_state_find` uses the policy's mark to select an SA, not an arbitrary packet mark. A single unmarked outbound policy would not preserve selection of the independently marked SAs. The actual kernel API therefore rules out that simplistic extension.

11. Give each edge a distinct IPsec request ID

Difference. Set the same unique local reqid on an edge's directional states and corresponding outbound per-edge policy template. A generic inbound template uses wildcard reqid zero. Keep the existing SPI, mark, direction, independent key and identity checks.

Why it can preserve behavior. Reqid is a local state-selection identifier, not a change to the ESP wire format or cryptographic identity. Linux's outbound SA hash uses addresses, family and reqid; using zero on every edge sends the host pair's states into the same lookup bucket. A distinct reqid partitions that lookup while the SPI and mark still select and authenticate the correct edge. Use this together with the selected inbound policy design.

Validation obligation. Repeat traffic and missing/wrong-key, wrong-SA/VNI, generation reuse and cleanup tests on the combined configuration. Reserve reqids by owned edge generation and do not reuse one until teardown ACKs. The final combined candidate passes these checks; the report retains the corresponding lifecycle and packet evidence.

[Linux XFRM state lookup implementation](#) and [policy lookup implementation](#).

12. Enable a sufficiently wide IPsec replay window

Difference. Current per-port IPsec leaves replay protection at its default disabled value. The selected cross-host candidate uses 4,096 packets, represented through XFRMA_REPLAY_ESN_VAL without enabling extended sequence numbers.

Why this change is needed. A finite window must accommodate normal packet reordering while rejecting old captured ciphertext. The 4,096-packet setting passed the final load and replay tests. It does not alter keys, SPI/mark binding or confidentiality.

Evidence and limits. The combined candidate repeats the full two-host packet proof, including an old ESP frame replayed after more than 4,096 packets, wrong key, wrong authenticated SA/VNI, total crypto removal and fresh-key recreation. The final 1,000-edge run passed all 2,000 directional pings, with zero sequence rejections and no XFRM error-counter increases after both hosts finished installation. Phase snapshots record 974 missing-SA packets on host 104 during concurrent installation, when the two sides are not yet jointly ready; that count stays unchanged through every ping and TCP sample. Product readiness must wait for both endpoint setup acknowledgements. No claim of zero cumulative installation drops is made. TCP throughput still varies by edge position and TCP retransmissions remain; those are reported separately. The window is a measured deployment choice, not a guarantee for arbitrary network reordering. Sequence exhaustion and coordinated fresh-key rekeying remain necessary. Do not silently reset sequence numbers under a reused key.

13. Use verified Netlink create echoes instead of redundant name lookups

Difference. Request NLM_F_ECHO on link creation and use the returned interface index where the real kernel provides it. Remove redundant pre-create lookups only in the authoritative clean-lease path. Existing-state reconciliation remains a separate operation.

Why it can be equivalent. The index comes from the acknowledged kernel create operation in that namespace. No cache survives across leases or namespaces. The prototype verified veth/bridge echoes against explicit name lookup and then exercised them in lifecycle, failure, concurrent reuse and real packet tests.

Important API boundary. Linux 6.12.95 returned no interface index for the VXLAN echo in the actual test, while a subsequent lookup returned the newly created device. The candidate therefore retains the VXLAN lookup. An ordinary ACK alone never supplies an index. Production must implement the verified per-device behavior and preserve error ACK handling; it must not infer indices or reuse a stale cache entry.

[Actual route-Netlink creation implementation.](#)

14. Admit encrypted traffic through the existing peer firewall, without a shared plaintext UDP allowance

Difference. The marked-IPsec path needs the existing reference-counted known-peer allowance for ESP. It does not need an entry mapping shared UDP 4790 to one peer in VXLAN_PORTS. Keep the plaintext-drop policy guards before exposure. Unencrypted VXLAN keeps its existing admission path.

Why this is required. VXLAN_PORTS currently maps one port to one peer, and its removal is per edge, so naively registering every encrypted shared-port edge would overwrite peer bindings and remove a shared entry prematurely. The existing eBPF program already permits ESP for known peers; PEERS already has per-edge reference counting. No widening of the static UDP allowlist or loss of per-edge cryptographic authentication is needed.

Evidence and implementation boundary. Every real-host marked-IPsec packet proof runs with only the two temporary PEERS entries; it does not add a UDP4790 VXLAN_PORTS entry. Wrong SA/VNI, missing crypto and unrelated-edge survival are tested. The current product call site still registers nondefault destination ports, so the future marked-encryption branch must explicitly skip that plaintext registration and preserve the existing PEERS lifetime handling. This is part of the implementation recipe, not a claim that product code was changed.